



Revisão Prova ED2 - pt1

1. Algoritmo de Dijkstra

O algoritmo de Dijkstra é uma abordagem gulosa (*greedy*) para encontrar o caminho de custo mínimo de um vértice de origem para todos os outros vértices em um grafo.

Informações Cruciais

- **Aplicação:** Problema de Caminho Mais Curto
- **Restrição Principal: Não suporta arestas com pesos negativos.** Se houver um peso negativo, o algoritmo pode retornar resultados incorretos, pois assume que, uma vez que um nó é visitado, seu custo mínimo foi definitivamente encontrado.
- **Estratégia:** Utiliza a técnica de relaxamento das arestas, sempre escolhendo o vértice não visitado com a menor distância acumulada.
- **Complexidade de Tempo:** Usando apenas arrays simples (como na implementação abaixo), a complexidade é $O(V^2)$, onde V é o número de vértices. É ideal para grafos densos.

Materiais de apoio:

https://youtu.be/EFg3u_E6eHU?si=Wxd6To-27RBAusap

<https://youtu.be/HUSolQwJ6cl?si=k7-brmqjk0X0fuE2>

<https://www.cs.usfca.edu/~galles/visualization/Dijkstra.html>

Implementação em Java

Implementação utilizando matriz de adjacências .

```
public class Dijkstra{
    public static int[] Dijkstra(int[][] matrizPesos) {
        int n = matrizPesos.length;
        int count = n;
        long[] dist = new long[n];
        int[] rota = new int[n];
        dist[0] = 0;
        rota[0] = -1;

        for (int i = 1; i < n; i++) {
            dist[i] = Integer.MAX_VALUE;
            rota[i] = -1;
        }

        boolean[] visitados = new boolean[n];

        while (count > 0) {
            count--;
            long min = Integer.MAX_VALUE;
            int v = -1;
            for (int i = 0; i < n; i++) {
                if (!visitados[i] && dist[i] != Integer.MAX_VALUE && dist[i] < min) {
                    min = dist[i];
                    v = i;
                }
            }
            visitados[v] = true;
        }
        return rota;
    }
}
```

```

        }
    }

    if (v == -1)
        break;

    vizitados[v] = true;
    for (int u = 0; u < n; u++) {
        if (matrizPesos[v][u] > 0 && !vizitados[u]) {
            if (dist[v] + matrizPesos[v][u] < dist[u]) {
                dist[u] = dist[v] + matrizPesos[v][u];
                rota[u] = v;
            }
        }
    }
}

return rota;
}

public static void imprimirCaminho(int[] rota, int origem, int destino) {

    if (rota[destino] == -1 && destino != origem) {
        System.out.println("Não existe caminho de " + origem + " até " + destino);
        return;
    }

    List<Integer> caminho = new ArrayList<>();
    int atual = destino;

    while (atual != -1) {
        caminho.add(atual);
        atual = rota[atual];
    }

    Collections.reverse(caminho);

    if (caminho.get(0) != origem) {
        System.out.println("Não existe caminho direto de " + origem + " até " + de
stino);
        return;
    }

    System.out.print("Caminho de " + origem + " até " + destino + ": ");
    for (int i = 0; i < caminho.size(); i++) {
        if (i == caminho.size() - 1) {
            System.out.println(caminho.get(i));
        } else {
            System.out.print(caminho.get(i) + " -> ");
        }
    }
}
}

```

2. Algoritmo de Bellman-Ford

O Bellman-Ford resolve o mesmo problema que o Dijkstra, mas utiliza a Programação Dinâmica. Ele é mais flexível, projetado para lidar com cenários onde o Dijkstra falha.

Informações Cruciais

- **Aplicação:** Problema de Caminho Mais Curto.
- **Vantagem Principal:** **Suporta arestas com pesos negativos** e é capaz de detectar **ciclos de peso negativo** (onde o peso total do ciclo é menor que zero, o que impossibilita a definição de um caminho mínimo finito).
- **Estratégia:** Relaxa todas as arestas do grafo exatamente $V - 1$ vezes (sendo V o número de vértices). Se no passo V uma distância ainda puder ser reduzida, significa que existe um ciclo negativo.
- **Complexidade de Tempo:** $O(V \cdot E)$, onde V é o número de vértices e E é o número de arestas. É mais lento que o Dijkstra, sendo preferido apenas quando há possibilidade de pesos negativos.

Materiais de apoio:

https://youtu.be/9PHkkOUavIM?si=DB_zxLvckX6wzw2l

<https://youtu.be/obWXjtgOL64?si=Z52IT08Pq9uNB4ev>

https://algorithms.discrete.ma.tum.de/graph-algorithms/spp-bellman-ford/index_en.html (aqui você visualiza cada iteração num desenho, enquanto vê o código)

Implementação em Java

Aqui, a representação mais simples é uma lista plana de arestas. Criamos uma classe estática `Aresta` e iteramos sobre um array simples.

Java

```
public class BellmanFord{

    static class Aresta{
        int origem, destino, peso;
        Aresta(int origem, int destino, int peso) {
            this.origem = origem;
            this.destino = destino;
            this.peso = peso;
        }
    }

    public void bellmanFord(int n, List<List<Vizinho>> grafo, int origem) {
        int[] distancias = new int[n];
        int[] rota = new int[n];

        for (int i = 0; i < n; i++) {
            distancias[i] = Integer.MAX_VALUE;
            rota[i] = -1;
        }
        distancias[origem] = 0;

        for (int k = 0; k < n; k++) {
            boolean altera = false;

            for (int u = 0; u < n; u++) {
```

```

        if (distancias[u] == Integer.MAX_VALUE)
            continue;

        for (Vizinho vizinho : grafo.get(u)) {
            int v = vizinho.destino;
            int peso = vizinho.peso;

            if (distancias[v] > distancias[u] + peso) {
                distancias[v] = distancias[u] + peso;
                rota[v] = u;
                altera = true;
            }
        }
    }

    if (!altera) {
        break;
    }

    if (k == n && altera) {
        System.out.println("Ciclo negativo detectado!");
    }
}
}
}

```

3. Tabela Comparativa de Decisão

Característica	Dijkstra	Bellman-Ford
Paradigma	Guloso (<i>Greedy</i>)	Programação Dinâmica
Pesos Negativos	Não suporta	Suporta perfeitamente
Ciclos Negativos	Não detecta	Detecta e alerta
Complexidade de Tempo	$O(V^2)$ (Array) ou $O(E + V \log V)$	$O(V \cdot E)$
Caso de Uso Ideal	Rotas de GPS, otimização padrão de redes	Arbitragem financeira, grafos com penalidades/descontos